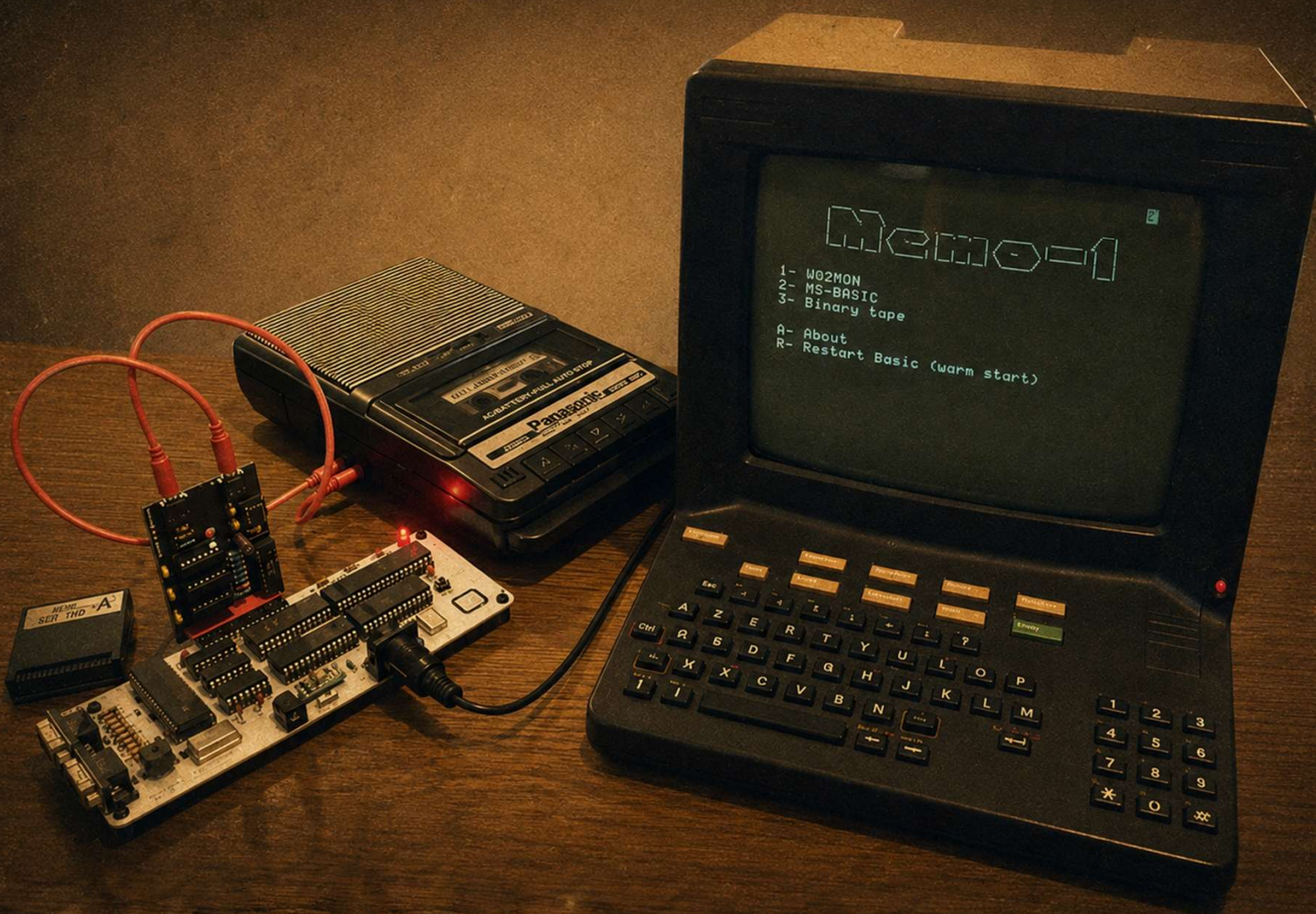


MEMO-1



Manuel de montage et d'utilisation

Ordinateur 65C02 : carte mère, extension mémoire et interface cassette

Le Memo-1, relié à un Minitel en guise de terminal et à un magnétophone via le module KCS.

Dépôt GitHub, code source, gerbers, BOM détaillées et mises à jour :
<https://github.com/MemoireMorte/Memo-1>

Par Benoît Aveline (Mémoire Morte), Licence Creative Commons BY-NC-SA 4.0

MEMO-1

Manuel de montage et d'utilisation

Dépôt GitHub, code source, gerbers, BOM détaillées et mises à jour :

<https://github.com/MemoireMorte/Memo-1>

Par Benoît Aveline (Mémoire Morte), Licence Creative Commons BY-NC-SA 4.0



534F59455A2043555249455558

1. Présentation

Le Memo-1 est un petit ordinateur à but pédagogique et ludique, construit autour d'un CPU 65C02 cadencé à 1MHz, associé à 32Ko de RAM et 16Ko de ROM. Il intègre un VIA 65C22 (manettes, son) et une ACIA 6551 (liaison série), et est pensé pour fonctionner avec un Minitel 1B comme écran et clavier.

Une extension mémoire de 8Ko de ROM peut être ajoutée dans le connecteur d'extension, et une interface cassette (KCS) permet de sauvegarder et charger des programmes, en version cassette audio classique ou avec un émulateur SD optionnel.

Le Minitel comme terminal

Le Memo-1 n'a ni écran ni clavier intégré : il utilise un Minitel comme terminal, relié à l'ACIA 6551 en liaison série. Un Minitel 1B minimum est nécessaire (le Minitel 1, plus ancien, ne convient pas). On reconnaît un 1B à la présence d'une touche « Fnct » sur son clavier, absente sur le Minitel 1.

La liaison se fait avec un câble MIDI (DIN-5), branché sur la prise péri-informatique du Minitel et sur le connecteur J3 du Memo-1.

Sur la plupart des Minitel 1B et modèles ultérieurs, cette prise péri-informatique est elle-même alimentée : le Memo-1 peut alors être mis sous tension directement par ce câble, sans alimentation séparée. Si votre Minitel ne fournit pas d'alimentation sur cette prise, utilisez le jack d'alimentation du Memo-1 (J5) avec une alimentation externe 7 à 12V CC, connecteur 5,5×2,1mm à centre positif.

Contenu du kit

- 1 PCB carte mère Memo-1
- 1 PCB carte d'extension mémoire (ROM extension)
- 1 PCB interface cassette KCS (+ module émulateur SD), voir le manuel dédié
- Ce manuel

⚠ Ce kit fournit les circuits imprimés et, selon la formule choisie, les composants associés. Consultez le dépôt [Github](#) pour la BOM complète avec liens fournisseurs, le schéma et les fichiers gerbers pour l'impression de nouveaux circuits.

2. Cartographie mémoire

Le 65C02 adresse 64Ko (0000-FFFF). Le décodage d'adresse du Memo-1 découpe cet espace par blocs de 4Ko, sur le quartet de poids fort (A15-A12) :

Adresses	Bloc(s)	Contenu
0000-7FFF	0x0-0x7	RAM (32Ko)
8000-8FFF	0x8	VIA (65C22)
9000-9FFF	0x9	ACIA (6551)
A000-BFFF	0xA-0xB	Connecteur d'extension (J4)
C000-FFFF	0xC-0xF	ROM (16Ko)

⚠ Le connecteur d'extension (J4) expose le bus CPU complet sur A000-BFFF (8Ko) : c'est là que se branche soit la carte d'extension mémoire (ROM), soit l'interface cassette KCS. Le menu de démarrage lit le premier octet à A000 pour détecter une ROM externe ; l'interface KCS utilise elle le registre B000 pour piloter la sortie cassette (voir manuel dédié).

VIA (65C22) : 8000-800B

Adresse	Registre
8000	Port B (lecture/écriture)
8001	Port A (lecture/écriture)
8002	Registre de direction des données, port B
8003	Registre de direction des données, port A
8004	Timer 1, octet de poids faible
8005	Timer 1, octet de poids fort
800B	Registre de contrôle auxiliaire

Le VIA ne déclenche pas d'interruption sur le Memo-1 ; il fournit les deux ports joystick Atari CX40 (voir la section Manettes de jeu) et la sortie son de l'instruction TONE.

ACIA (6551) : 9000-9003

Adresse	Registre
9000	Registre de données
9001	Registre d'état
9002	Registre de commande
9003	Registre de contrôle

⚠ La 6551 est indispensable ici : une W65C51 ne fonctionne pas correctement avec le Minitel (incompatibilités matérielles connues).

3. Avant de commencer

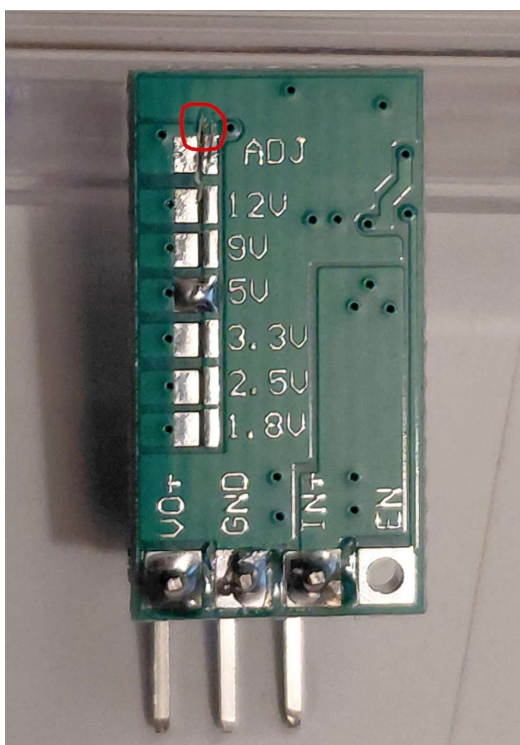
Choix de la puce ROM

La ROM 28C256 peut être remplacée par une 27128, à condition de laisser la broche 1 déconnectée (la lever hors du support). Avec une 27128, utilisez le binaire 16K ; avec une 28C256, utilisez le binaire 32K (mêmes données, décalées de 16K).

Réglage du convertisseur DC-DC

Avant de souder le convertisseur DC-DC, réglez-le pour délivrer 5V, par l'une des deux méthodes suivantes :

1. Alimentez le convertisseur seul (hors circuit) et ajustez la petite vis tout en mesurant la tension au multimètre.
2. Coupez le pont ADJ au dos du module et faites un pont de soudure sur la position « 5V ». Vérifiez bien qu'il n'y a plus de continuité sur les pastilles ADJ.



Réglage du convertisseur DC-DC en position 5V.

⚠ Ne réglez jamais le convertisseur en circuit, avec les autres composants déjà soudés.

4. Nomenclature (BOM)

Les tableaux ci-dessous listent les composants de chaque carte. Les références fournisseurs complètes (liens d'achat) sont disponibles dans [how-to/BOM_Rev1.csv](#) sur le dépôt [GitHub](#).

4.1 Carte mère

Réf.	Qté	Valeur / Type	Description
BZ1	1	Buzzer	Buzzer piézo passif
C1,C3-C8	7	0.1µF	Condensateur céramique (104)
C2	1	30pF	Condensateur céramique (300)
D1	1	LED	LED 5mm, témoin d'alimentation
J1,J2	2	DSUB-9	Ports joystick Atari CX40
J3	1	DIN-5	Connecteur MIDI / DIN-5 femelle (vers Minitel)
J4	1	Ext.	Barrette 2×16 broches, 2,54mm, soudée
J5	1	DC IN	Jack d'alimentation femelle 5,5×2,1mm
R1,R2,R4,R17	4	1kΩ	Résistance 1/4W 5%
R3	1	1MΩ	Résistance 1/4W 5%
R5-R14	10	10kΩ	Résistance 1/4W 5%
R15,R18	2	4,7kΩ	Résistance 1/4W 5%
R16	1	330Ω	Résistance 1/4W 5%
SW1	1	RESET	Bouton poussoir 6×6
SW2	1	Alim.	Interrupteur à glissière SPDT
U1	1	CPU	W65C02S (ou R65C02)
U2	1	VIA	W65C22S (ou 6522 générique)
U3	1	RAM	HM62256B (ou AS6C62256)
U4	1	ROM	28C256 (ou 27128, voir note ROM)
U5	1	NAND	74LS00
U6	1	OR	74LS32
U7	1	AND	74LS08
U8	1	ACIA	6551 (pas de W65C51, incompatible Minitel)
U9	1	5V	Convertisseur DC-DC step-down (ou régulateur 7805)
Y1	1	1,8432MHz	Quartz
Y2	1	1MHz	Oscillateur DIP-14
-	1	Câble	Câble DIN-5, 5 broches câblées
U1,U2	2	Support	Support DIP-40 large (recommandé)
U3,U4,U8	3	Support	Support DIP-28 large (recommandé)
U5,U6,U7	3	Support	Support DIP-14 étroit (recommandé)

4.2 Carte d'extension mémoire

Réf.	Qté	Valeur / Type	Description
C1	1	0.1µF	Condensateur céramique
U1	1	ROM	28C64
J1	1	Ext.	Barrette femelle 2×16, 2,54mm
U1	1	Support	Support DIP-28 large (recommandé)

⚠ La carte d'interface cassette KCS (BOM, montage et usage) fait l'objet d'un manuel séparé.

5. Montage : carte mère et extension

Étape 1 : Composants passifs

1. Résistances, à souder en premier.
2. Supports de circuits intégrés : vérifiez le sens du détrompeur sur la sérigraphie. N'insérez pas encore les CI.
3. Condensateurs.

Étape 2 : Composants d'alimentation

1. Connecteur DIN-5 en J3.
2. Jack d'alimentation en J5.
3. Interrupteur d'alimentation en SW2.
4. Convertisseur DC-DC en U9, orientation carte en lecture normale : 1ère broche (EN) non soudée, 2ème (IN+) sur la pastille gauche, 3ème (GND) au centre, 4ème (VO+) à droite.
5. LED en D1.

Étape 3 : Test d'alimentation

Alimentez la carte via le jack ou le DIN-5 (avec un Minitel). La LED doit s'allumer.

⚠ Si la LED ne s'allume pas : vérifiez la position de l'interrupteur, la soudure de U9, la tension d'entrée sur IN+/GND, la tension de sortie sur VO+/GND (doit faire 5V), et la polarité de la LED.

Étape 4 : Assemblage final

1. Une fois le 5V confirmé, soudez tous les composants restants.
2. Avant d'insérer les CI : revérifiez l'alimentation et toutes les soudures.
3. Insérez les circuits intégrés dans leurs supports. Attention, le CPU se place à l'envers par rapport aux autres CI. Vérifiez que toutes les broches sont bien engagées.

Carte d'extension mémoire

Les barrettes femelles se soudent côté composants et la carte mère utilise des broches mâles coudées à 90°. Programmez la ROM 28C64 avant de l'insérer.

⚠ Pour vérifier que la carte d'extension fonctionne, vous pouvez la programmer avec le binaire de démonstration `stillAlive.bin` fourni dans `how-to/` sur [GitHub](#).

6. Test et première mise en route

1. Alimentez la carte (jack ou DIN-5 via le Minitel).
2. Vérifiez que la LED d'alimentation s'allume.
3. Reliez le Minitel 1B au Memo-1 avec le câble DIN-5, puis allumez le Minitel.
4. Appuyez sur le bouton RESET du Memo-1.
5. Le menu de démarrage apparaît sur l'écran du Minitel.

Le menu propose : « 1 » pour lancer WOZMON (moniteur mémoire), « 2 » pour démarrer MS-BASIC, « 3 » pour le sous-menu de sauvegarde/chargement cassette (voir le manuel dédié à l'extension KCS), « 4 » pour exécuter le code de la ROM d'extension (si détectée), et « A » pour l'écran « à propos ».

7. Utilisation

WOZMON

WOZMON est le moniteur mémoire écrit par Steve Wozniak pour l'Apple 1, repris ici tel quel (src/wozmon.s sur GitHub). Il permet d'examiner et de modifier la mémoire octet par octet, et de lancer l'exécution de code machine (pratique pour du debug bas niveau ou pour charger un petit programme à la main). Toutes les adresses et valeurs se saisissent en hexadécimal, sans préfixe (par exemple 0200, pas \$0200 ni 0x0200, ou encore 200 puisque les zéros non significatifs sont facultatifs).

Commande	Effet
AAAA	Affiche le contenu de l'adresse AAAA
AAAA.BBBB	Affiche le contenu de la plage AAAA à BBBB
AAAA: XX YY ...	Dépose les octets XX, YY... à partir de AAAA
AAAA R	Exécute le code machine à partir de AAAA

Exemple 1 : lire et modifier un octet

Dans les exemples suivants, pour des raisons de lisibilité, ce qui est écrit par **l'ordinateur est en vert**, et ce qui est écrit par **l'utilisateur est en bleu**. Ces couleurs ne sont pas présentes sur l'écran du Memo-1.

On examine l'octet à l'adresse 0000, on le modifie, puis on vérifie le résultat :

```
\
0000
0000: A5
0000: FF
0000: A5
0000
0000: FF
```

⚠ Après un dépôt (« : »), le moniteur réaffiche d'abord l'adresse avec son ancien contenu, avant de stocker les nouvelles valeurs. C'est un comportement hérité de l'Apple 1, tout à fait normal.

Exemple 2 : écrire et exécuter un mini-programme

On dépose en RAM (à partir de 0200) un petit programme qui écrit le caractère « A » directement dans le registre de données de l'ACIA (\$9000), puis retourne :

```
LDA #$41    ; A9 41    charge le caractère 'A'
STA $9000   ; 8D 00 90 écrit sur le port série (ACIA)
RTS         ; 60      retour
```

Sur le terminal WOZMON :

```
\
0200: A9 41 8D 00 90 60
0200: EA
0200.0205
0200: A9 41 8D 00 90 60
0200R
A
```

La ligne 0200.0205 permet de vérifier que les 6 octets ont bien été déposés avant de lancer le programme avec 0200R ; le « A » qui s'affiche est produit par le programme lui-même.

C'est quoi ce charabia ?

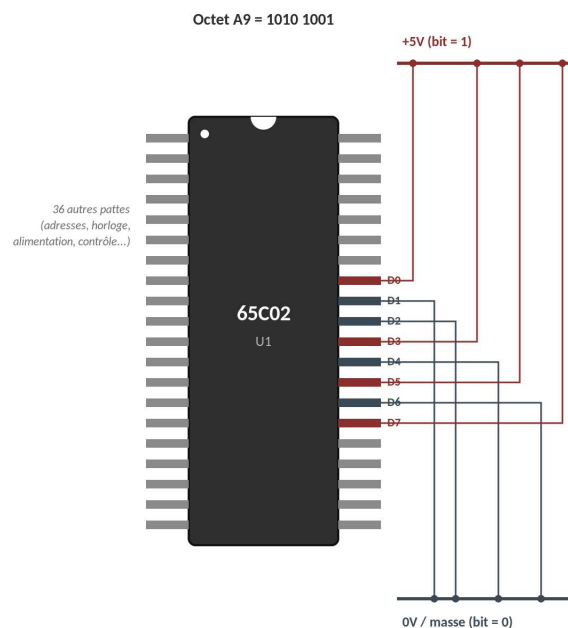
Une ligne comme A9 41 8D 00 90 60 ressemble à du charabia. Voici ce qui se cache derrière : une petite digression pour comprendre ce qu'on manipule vraiment.

Le processeur ne « comprend » rien au sens humain du terme : il réagit uniquement à la tension électrique présente sur ses pattes (ses broches), à chaque battement de l'horloge. Selon la combinaison de tensions reçues, un circuit câblé en dur à l'intérieur de la puce déclenche une action précise et prédéfinie : charger une valeur, l'additionner, la copier ailleurs, sauter à une autre adresse...

Pour représenter cette tension (présente ou absente), on utilise deux chiffres : 1 (tension) et 0 (pas de tension). Compter avec seulement deux chiffres s'appelle le binaire. Un octet est un paquet de 8 de ces 0 et 1, soit 256 combinaisons possibles, assez pour coder toutes les actions que sait faire le processeur.

Écrire une adresse ou une instruction en binaire (10101001, par exemple) est long et peu lisible. On regroupe donc les bits par paquets de 4, que l'on représente avec un seul chiffre hexadécimal (0 à 9, puis A à F). Le même octet s'écrit alors simplement A9 : deux caractères au lieu de huit.

Chaque valeur comme A9 correspond, câblée en dur dans le processeur, à une action bien précise. Pour se représenter la chose simplement : on peut imaginer huit des pattes du boîtier (le bus de données) reliées en permanence soit au +5V, soit à la masse (0V), selon les 8 bits de l'octet reçu. La logique interne, elle aussi câblée en dur, lit cette combinaison et déclenche l'action correspondante :



Vue simplifiée d'un boîtier DIP-40 : les 8 broches du bus de données figées à 0V ou 5V représentent l'octet A9. En réalité, le bus change à chaque cycle d'horloge plutôt que d'être câblé en dur, mais l'idée, une tension qui code un bit, reste la même.

On appelle A9 un opcode (« code d'opération »). Retenir 256 codes numériques par cœur serait fastidieux, alors on leur a donné des petits noms courts et mnémotechniques : LDA (Loa**D** Accumulator), STA (ST**o**re Accumulator), RTS (Re**T**urn from Subroutine)... C'est ça, l'assembleur : un langage qui associe un nom lisible à chaque opcode. Un programme, lui aussi appelé assembleur, se charge ensuite de retraduire ces noms en opcodes que le processeur peut exécuter. C'est exactement ce que fait ca65 pour le code source du Memo-1 (voir le README sur GitHub).

La table complète des 256 opcodes du 65C02 se trouve en Annexe A, en fin de manuel.

MS-BASIC

BASIC est un langage dit de haut niveau : ses instructions (PRINT, GOTO, IF...THEN, FOR...NEXT) ressemblent à des phrases et manipulent des concepts (des variables, des boucles, du texte) plutôt que des registres ou des adresses mémoire précises comme en assembleur. Une seule ligne comme PRINT "HELLO" peut correspondre, une fois traduite, à des dizaines d'instructions machine du genre de celles vues plus haut (LDA, STA...) : c'est ce qui le rend bien plus rapide à écrire, au prix d'un peu de performance à l'exécution.

Autre différence avec l'assembleur : l'assembleur est compilé, c'est-à-dire traduit une bonne fois pour toutes en opcodes avant l'exécution (c'est le travail de ca65). BASIC, lui, est interprété : chaque ligne est lue, traduite et exécutée à la volée à chaque lancement, par un programme qu'on appelle justement un interpréteur. C'est plus lent qu'un programme assemblé, mais ça permet d'écrire une ligne, de taper RUN, et de voir le résultat tout de suite, sans étape de compilation séparée.

Petite note d'histoire : cet interpréteur n'est pas anodin. Sa lignée remonte au tout premier produit jamais vendu par Microsoft, en 1975 : un interpréteur BASIC écrit par Bill Gates, Paul Allen et Monte Davidoff pour l'Altair 8800, le micro-ordinateur en kit qui a lancé toute une industrie. La version portée sur 6502 (BASIC-M6502, avec Ric Weiland), désassemblée par Michael Steil, est celle qui tourne aujourd'hui sur le Memo-1, cinquante ans plus tard.

Un premier exemple :

```
10 PRINT "HELLO MEMO-1"
20 GOTO 10
RUN
```

Quelques bases pour découvrir le langage, sans prétendre à l'exhaustivité :

Numérotation des lignes : chaque instruction commence par un numéro (10, 20, 30... par convention, pour pouvoir en insérer entre deux plus tard). Le programme s'exécute dans l'ordre croissant des numéros, sauf instruction de saut comme GOTO ou GOSUB.

Variables : pas besoin de les déclarer : elles existent dès qu'on les utilise. Une lettre, éventuellement suivie d'un chiffre, comme A ou B1, désigne un nombre ; si le nom se termine par \$, comme NOM\$, c'est une chaîne de caractères. A = 10 affecte 10 à A (le mot LET est optionnel devant : LET A = 10 fonctionne aussi).

Afficher, boucler, tester : PRINT affiche du texte entre guillemets ou la valeur d'une variable (plusieurs éléments séparés par ; ou ,) ; FOR...TO / NEXT répète un bloc d'instructions un nombre de fois donné ; IF...THEN exécute une instruction seulement si une condition est vraie.

```
10 FOR I = 1 TO 5
20 PRINT "TOUR NUMERO "; I
30 NEXT I
40 END
RUN
```

La liste complète des instructions, fonctions et opérateurs disponibles sur le Memo-1 se trouve en Annexe B, en fin de manuel.

Manettes de jeu

Deux ports joystick Atari CX40 sont disponibles via le VIA. La fonction BASIC JOY(0) et JOY(1) renvoie l'état de chaque port, sur 5 bits (haut, bas, gauche, droite, tir) :

```
10 A = JOY(0)
20 IF A = 31 THEN GOTO 10
```

```
30 IF A = 30 THEN PRINT "UP"  
40 IF A = 29 THEN PRINT "DOWN"  
50 IF A = 27 THEN PRINT "LEFT"  
60 IF A = 23 THEN PRINT "RIGHT"  
70 IF A = 15 THEN PRINT "FIRE"  
80 GOTO 10  
RUN
```

Son

L'instruction BASIC TONE joue une onde carrée sur le port B du VIA (pratique pour des bips ou de petites mélodies). Exemple : jouer un LA (440Hz, valeur 1136) pendant un court instant :

```
10 TONE 1136  
20 K = 40  
30 FOR I=0 TO K STEP 1  
40 PRINT I  
50 NEXT I  
60 TONE 0  
70 END  
RUN
```

Le tableau de correspondance note/valeur pour les 12 demi-tons se trouve en Annexe C, en fin de manuel.

Sauvegarde et chargement

La sauvegarde et le chargement de programmes via l'interface cassette KCS, depuis BASIC (LOAD/SAVE) ou depuis le menu de démarrage pour un programme en langage machine, avec un vrai lecteur cassette ou avec l'émulateur SD, sont couverts par le manuel dédié.

8. Dépannage rapide

La LED ne s'allume pas

- Vérifiez la position de l'interrupteur d'alimentation.
- Vérifiez la soudure et l'orientation du convertisseur DC-DC (U9).
- Mesurez la tension d'entrée sur IN+/GND, puis la sortie sur VO+/GND (doit être 5V).
- Vérifiez la polarité de la LED.

Rien ne s'affiche sur le Minitel

- Vérifiez le câble DIN-5 (toutes les broches doivent être câblées).
- Rappuyez sur RESET après avoir allumé le Minitel.
- Vérifiez l'orientation des circuits intégrés, en particulier le CPU (à l'envers par rapport aux autres).

Aucune ROM d'extension détectée

- Vérifiez que la ROM est bien programmée et insérée dans le bon sens.
- Vérifiez la continuité du connecteur d'extension entre les deux cartes.

9. Ressources et communauté

Le dépôt GitHub regroupe le code source, les fichiers gerbers, les BOM détaillées avec liens fournisseurs, les schémas, ainsi que les mises à jour du projet :

<https://github.com/MemoireMorte/Memo-1>

Remerciements

- Ben Eater, conception d'ordinateur 6502 et tutoriels (licence CC-BY)
- Ian Ward, vidéos YouTube sur l'écran LCD 2004 et le timer 555
- Wozmon, par Steve Wozniak
- BASIC-M6502 (Weiland & Gates) et sa désassemblage par Michael Steil

Memo-1, par Benoît Aveline (Mémoire Morte), © 2025, Creative Commons BY-NC-SA 4.0

Annexe A : Table des opcodes du 65C02

Cette table liste le mnémonique associé à chacun des 256 opcodes du 65C02 (ligne = poids fort de l'octet, colonne = poids faible). Un seul code est officiellement l'instruction NOP (« ne rien faire ») : \$EA. Les cases marquées X sont des opcodes réservés, non attribués à une instruction. WDC garantit, dans la fiche technique du W65C02S, qu'ils exécutent eux aussi un NOP sans effet de bord (juste plus ou moins d'octets et de cycles consommés selon le code), mais ce n'est pas une instruction standard au même titre que les autres : d'autres puces de la famille 65C02 (Rockwell, notamment) utilisent certaines de ces cases réservées pour des instructions différentes, comme RMB/SMB/BBR/BBS ci-dessous.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0x	BRK	ORA	X	X	TSB	ORA	ASL	RMB0	PHP	ORA	ASL	X	TSB	ORA	ASL	BBR0
1x	BPL	ORA	ORA	X	TRB	ORA	ASL	RMB1	CLC	ORA	INC	X	TRB	ORA	ASL	BBR1
2x	JSR	AND	X	X	BIT	AND	ROL	RMB2	PLP	AND	ROL	X	BIT	AND	ROL	BBR2
3x	BMI	AND	AND	X	BIT	AND	ROL	RMB3	SEC	AND	DEC	X	BIT	AND	ROL	BBR3
4x	RTI	EOR	X	X	X	EOR	LSR	RMB4	PHA	EOR	LSR	X	JMP	EOR	LSR	BBR4
5x	BVC	EOR	EOR	X	X	EOR	LSR	RMB5	CLI	EOR	PHY	X	X	EOR	LSR	BBR5
6x	RTS	ADC	X	X	STZ	ADC	ROR	RMB6	PLA	ADC	ROR	X	JMP	ADC	ROR	BBR6
7x	BVS	ADC	ADC	X	STZ	ADC	ROR	RMB7	SEI	ADC	PLY	X	JMP	ADC	ROR	BBR7
8x	BRA	STA	X	X	STY	STA	STX	SMB0	DEY	BIT	TXA	X	STY	STA	STX	BBS0
9x	BCC	STA	STA	X	STY	STA	STX	SMB1	TYA	STA	TXS	X	STZ	STA	STZ	BBS1
Ax	LDY	LDA	LDX	X	LDY	LDA	LDX	SMB2	TAY	LDA	TAX	X	LDY	LDA	LDX	BBS2
Bx	BCS	LDA	LDA	X	LDY	LDA	LDX	SMB3	CLV	LDA	TSX	X	LDY	LDA	LDX	BBS3
Cx	CPY	CMP	X	X	CPY	CMP	DEC	SMB4	INY	CMP	DEX	WAI	CPY	CMP	DEC	BBS4
Dx	BNE	CMP	CMP	X	X	CMP	DEC	SMB5	CLD	CMP	PHX	STP	X	CMP	DEC	BBS5
Ex	CPX	SBC	X	X	CPX	SBC	INC	SMB6	INX	SBC	NOP	X	CPX	SBC	INC	BBS6
Fx	BEQ	SBC	SBC	X	X	SBC	INC	SMB7	SED	SBC	PLX	X	X	SBC	INC	BBS7

⚠ Ce tableau donne les mnémoniques, pas les modes d'adressage ni les temps d'exécution (un même mnémonique comme LDA peut correspondre à plusieurs opcodes selon la façon dont l'adresse est fournie). Pour le détail complet (y compris la liste exacte des codes X avec leur nombre d'octets et de cycles), voir la fiche technique du W65C02S ou 6502.org.

Annexe B : Commandes du BASIC

Liste des instructions, fonctions et opérateurs de l'interpréteur BASIC tel que configuré sur le Memo-1 (src/token.s sur GitHub).

Instructions

Commande	Effet
CLEAR	Réinitialise les variables
CONT	Reprend l'exécution après STOP
DATA	Définit des valeurs à lire avec READ
DEF	Définit une fonction utilisateur (DEF FN)
DIM	Déclare un tableau (variable indexée)
END	Termine l'exécution du programme
FOR / NEXT	Boucle répétitive (avec TO et STEP)
GET	Lit un caractère au clavier sans attendre Entrée
GOSUB / RETURN	Appelle un sous-programme et y revient
GOTO	Saute à un numéro de ligne
IF...THEN	Exécute une instruction si une condition est vraie
INPUT	Demande une valeur à l'utilisateur
LET	Affecte une valeur à une variable (optionnel)
LIST	Affiche le programme en mémoire
LOAD / SAVE	Charge / sauvegarde un programme (interface KCS)
NEW	Efface le programme en mémoire
ON	Saut conditionnel multiple (ON x GOTO/GOSUB ...)
POKE	Écrit un octet en mémoire
PRINT	Affiche du texte ou des valeurs
READ	Lit la valeur suivante d'une instruction DATA
REM	Commentaire, ignoré à l'exécution
RESTORE	Remet le pointeur DATA au début
RUN	Exécute le programme depuis le début
STOP	Interrompt l'exécution (reprise avec CONT)
TONE	Joue une note sur le haut-parleur (spécifique Memo-1)
WAIT	Attend qu'une adresse mémoire remplisse une condition

Fonctions

Fonction	Effet
ABS(x)	Valeur absolue de x
ASC(a\$)	Code ASCII du premier caractère
CHR\$(x)	Caractère correspondant à un code ASCII
COS(x) / SIN(x) / TAN(x) / ATN(x)	Fonctions trigonométriques
EXP(x)	Exponentielle de x
FRE(x)	Mémoire libre restante

Fonction	Effet
INT(x)	Partie entière de x
JOY(port)	État d'un joystick (spécifique Memo-1)
LEFT\$(a\$,n) / RIGHT\$(a\$,n)	Extrait les n premiers / derniers caractères
LEN(a\$)	Longueur d'une chaîne
LOG(x)	Logarithme népérien de x
MID\$(a\$,n,l)	Extrait l caractères à partir de la position n
PEEK(adresse)	Lit un octet en mémoire
POS(x)	Position du curseur
RND(x)	Nombre aléatoire
SGN(x)	Signe de x (-1, 0 ou 1)
SQR(x)	Racine carrée de x
STR\$(x)	Convertit un nombre en chaîne
USR(x)	Appelle une routine en langage machine
VAL(a\$)	Convertit une chaîne en nombre

Opérateurs et mots-clés

Symbole	Effet
+ - * / ^	Opérations arithmétiques (addition, soustraction, multiplication, division, puissance)
= < >	Comparaisons (égal, inférieur, supérieur, combinables : <=, >=, <>)
AND OR NOT	Opérateurs logiques
FN	Appelle une fonction définie avec DEF FN
TAB(SPC(	Positionnement du curseur dans une instruction PRINT
THEN	Utilisé avec IF pour introduire l'action à exécuter
TO STEP	Utilisés avec FOR...NEXT pour définir la plage et le pas

Annexe C : Table des notes pour TONE

Fréquences et valeurs à passer à l'instruction TONE pour jouer chacune des 12 notes de la gamme chromatique, à 1MHz (voir la section Son, p. Utilisation).

Note	Fréquence	Valeur TONE
DO	261,63 Hz	1911
DO#	277,18 Hz	1808
RE	293,66 Hz	1702
RE#	311,13 Hz	1607
MI	329,63 Hz	1517
FA	349,23 Hz	1432
FA#	369,99 Hz	1350
SOL	392,00 Hz	1275
SOL#	415,30 Hz	1203
LA	440,00 Hz	1136
LA#	466,16 Hz	1073
SI	493,88 Hz	1010